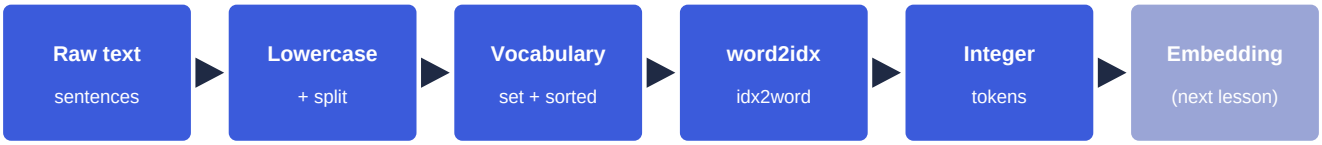
Text to Token: How It Works and Why It Matters

Lesson 02, Turning Text into Numeric Tokens (01 - Text to Vectors Basics)



1. What tokenization is

A neural network only does arithmetic, so it cannot read the string *"to be or not to be"*. **Tokenization** is the step that converts text into a sequence of integers (tokens) and back. The lesson builds the simplest version: **one lowercase word = one token**, using a vocabulary of unique words and two dictionaries that are exact inverses of each other.

2. The full working, step by step

Step	What the code does	Result on the 3-sentence corpus
1. Corpus	`text = [...]` holds three short quotations	3 sentences
2. Normalize	`' '.join(text).lower()` merges and lowercases so *Be* and *be* match	one lowercase string
3. Split	`re.split(r'\s', ...)` cuts at every whitespace character	`allwords`: 28 words
4. Vocabulary	`sorted(set(allwords))` removes duplicates, then orders them alphabetically	`vocab`: 20 unique words
5. Lookup tables	`enumerate(vocab)` builds `word2idx` and `idx2word`	all=0, already=1, ... yourself=19
6. Encode	`[word2idx[w] for w in allwords]`	[0, 13, 17, 2, 6, 14, 11, ...]
7. Decode	`[idx2word[i] for i in ids]`	original words, lowercase

Why `sorted`? A Python `set` has no guaranteed order, so without sorting the same text could receive different IDs on different runs. Sorting makes the mapping deterministic. **Why two dictionaries?** A dictionary looks up in one direction only; `word2idx` encodes and `idx2word` decodes, and the round trip `decoder(encoder(x))` is the basic correctness test.

3. Code explained

```
import re
text = ['All that we are is the result of what we have thought',
        'To be or not to be that is the question',
        'Be yourself everyone is already taken']

allwords = re.split(r'\s', ' '.join(text).lower())   # 28 tokens (with repeats)
vocab    = sorted(set(allwords))                      # 20 unique, alphabetical

word2idx = {w: i for i, w in enumerate(vocab)}        # notebook uses an explicit loop
idx2word = {i: w for i, w in enumerate(vocab)}

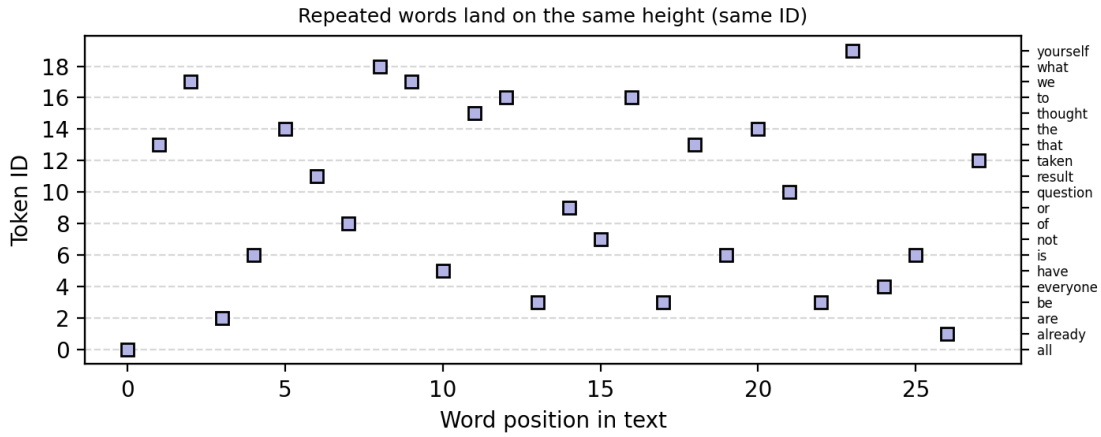
def encoder(text):
    return [word2idx[w] for w in re.split(' ', text.lower())]

def decoder(indices):
    return [idx2word[i] for i in indices]

encoder('to be')            # [16, 3]
decoder(encoder('to be'))   # ['to', 'be']
```

- **Encoder:** lowercase, split on spaces, look every word up in `word2idx`. Unknown words raise a `KeyError`.
- **Decoder:** look every integer up in `idx2word`. It returns a list, so use `' '.join(...)` for a sentence.
- **Fake quotes:** `np.random.randint` picks random IDs that decode to word salad. That is a language model with no knowledge at all, which shows what training must improve.

4. Seeing the tokens



Each square is one word of the text. Same height means same word: *be* (ID 3) and *is* (ID 6) each appear three times. A text is just a sequence of integers, and repetition becomes visible structure.

5. Context around a token

The notebook finds every position of *to* with `np.where` and prints the token before and after it: `[15, 16, 3]` ("thought to be") and `[7, 16, 3]` ("not to be"). The ID of *to* is identical (16), but its neighbours differ. This is a preview of how language models learn: a token is understood by the company it keeps. Edge case: at index 0, `tokens[t-1:t+2]` becomes `tokens[-1:2]`, an empty slice.

6. Significance in an LLM

- **It is the gateway.** Every LLM stage (embeddings, attention, output layer) assumes text is already numeric, and generated numbers are decoded back to text by the same table.
- **Vocabulary size sets model size.** The input embedding table and the output layer each have one row or column per token, so a bigger vocabulary means more parameters and a larger softmax.
- **Tokenization sets sequence length.** Finer pieces mean more tokens per sentence, which means more compute and a smaller effective context window.
- **It must be consistent.** The same tokenizer and vocabulary must be used for training and inference. Change the corpus and IDs shift, so save the vocabulary with the data.
- **IDs are labels, not quantities.** The number 16 is not "bigger" than 3; it is just alphabetical position. Giving numbers meaning is the job of embeddings (lesson 03).

7. Important things to remember

Weakness of this tokenizer	Example (verified by running the code)	Real-world fix
Unknown words crash it	`encoder('hello')` raises KeyError: 'hello'	Special <UNK> token, or subword pieces
Punctuation sticks to words	`encoder('to be, or')` raises KeyError: 'be,'	Split punctuation with a regex or tokenizer rules
Fragile whitespace	trailing or double space gives an empty string, KeyError: ''	`text.split()` collapses whitespace
Lowercasing loses information	*Apple* and *apple* become one token	Case-sensitive or byte-level tokenizers
Huge vocabulary on real text	every word form is a separate entry	Subword methods such as Byte Pair Encoding

From toy to real: production LLMs keep the same two-table idea (token to ID and ID to token) but split text into **subwords** (for example *token* + *ization*), add special tokens for padding and end-of-text, and learn the vocabulary from a large corpus with tens of thousands of entries or more. Words are too many, characters make sequences too long, and subwords sit in the middle.

Takeaway: tokenization is reversible, deterministic text-to-integer conversion. Master this small version and the real ones, and the embeddings that follow, become straightforward.

Text to Token, Part 2: Tokenizing a Whole Novel

Lesson 02, Turning Text into Numeric Tokens (01 - Text to Vectors Basics) · notebook `text2tokens_intermediate.ipynb`

Raw book 446,582 chars ▶ **Clean** ASCII, no digits, lower ▶ **Split** 73,888 words ▶ **Vocabulary** 7,243 unique ▶ **Encode/Decode** round trip OK ▶ **<|unk|>** unknown words

1. What changed from the toy version

Part 1 tokenized **three sentences** (28 words, 20-word vocabulary). Part 2 applies the same idea, one lowercase word = one token, to **a complete novel**: *Frankenstein* by Mary Shelley (Project Gutenberg #84). Real text forces three new problems to the surface: it must be **cleaned** before splitting, its vocabulary has a very **skewed frequency distribution**, and a tokenizer built from it **breaks on words it has never seen**.

2. The full working, step by step

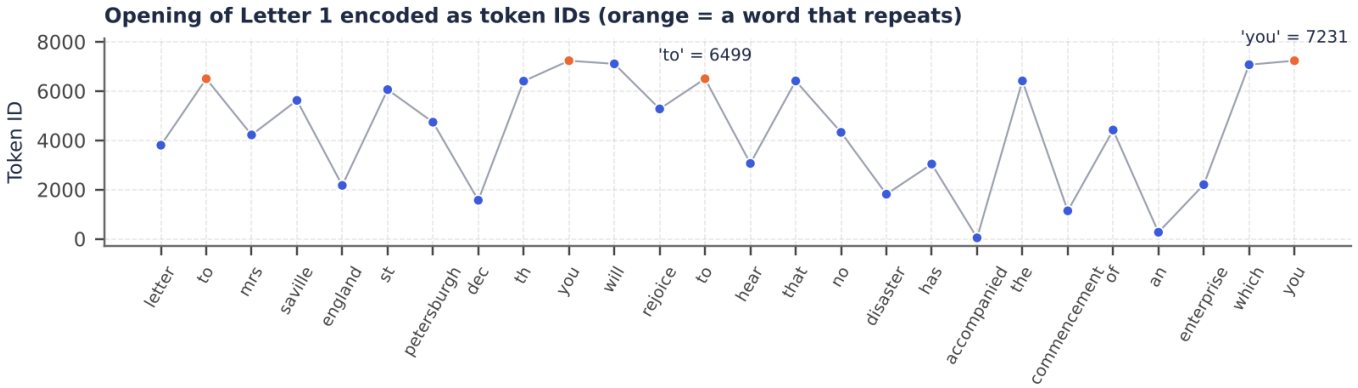
Step	What the code does	Result on the novel
1. Download	`requests.get(url).text` fetches the plain-text eBook	446,582 characters (incl. licence header)
2. Fix artefacts	replace `\r\n`, mis-decoded (mojibake) quotes and dashes, and _ with spaces	line breaks and italics markers gone
3. Normalise	`re.sub(r'[^\x00-\x7F]+', ' ', ...)`, `re.sub(r'\d+', '', ...)`, `.lower()`	ASCII-only, no digits, one case
4. Split	`re.split(rf'[{string.punctuation}\s]+', text)`, drop empty and 1-letter items	words: 73,888
5. Vocabulary	`sorted(set(words))`, then `word2idx` / `idx2word`	vocab: 7,243 (abandon = 0 ... zeal = 7242)
6. Encode / decode	NumPy loop fills an `int` array; decoder joins words	whole book round trip: True
7. Random text	`randint` (uniform) and `cumsum(choice([1,-1],50))` (Brownian walk)	word salad; alphabetical neighbours
8. Statistics	word-length counts, `Counter` frequencies, vocabulary growth	mean length 4.65; 42% of words used once
9. New sentence	same split on `'Hi I chatgpt am going home this week'`	`KeyError: 'hi'` (3 of 8 unknown)
10. <|unk|>	add special token at ID `len(word2idx)` and fall back to it	[7243 7243 7243 246 2879 3150 6444 7034]

3. Code explained

```
text = re.sub(r'[^\x00-\x7F]+', ' ', text)        # drop non-ASCII (curly quotes, dashes)
text = re.sub(r'\d+', '', text).lower()          # drop digits, lowercase
puncts4re = rf'[{string.punctuation}\s]+'          # any run of punctuation/whitespace
words = [w for w in re.split(puncts4re, text) if len(w.strip()) > 1]
vocab = sorted(set(words))                          # 7,243 unique, alphabetical
word2idx = {w: i for i, w in enumerate(vocab)};  idx2word = {i: w for i, w in enumerate(vocab)}

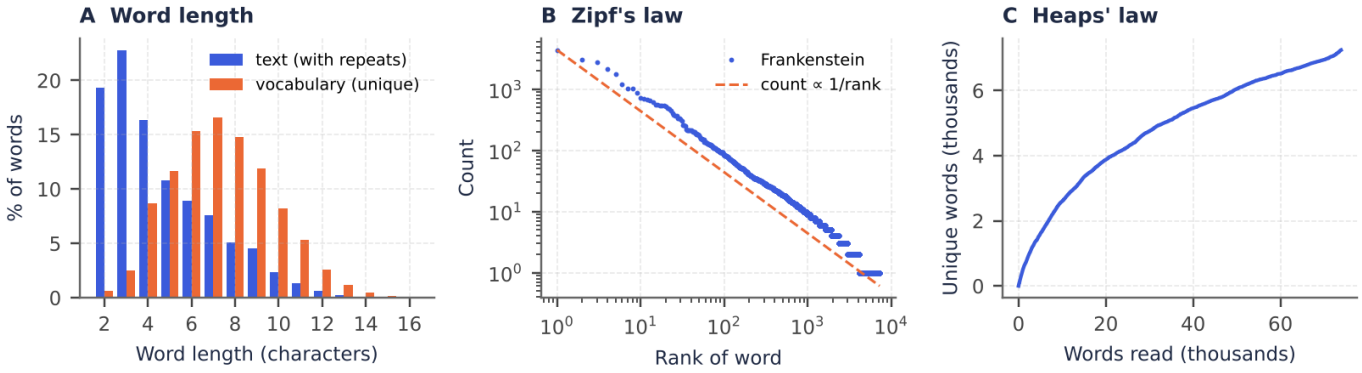
word2idx_new['<|unk|>'] = len(word2idx)             # next free ID = 7243 (no gap)
idxs[i] = encode_dict[w] if w in encode_dict else encode_dict['<|unk|>']   # never crashes
```

`rf` = raw f-string, so `\s` and `\d` reach the regex engine untouched (no escape warnings). Dropping 1-letter items removes the stray *s* of *victor's*, but also the real words *i* and *a*, a trade-off that resurfaces in step 9.



Each dot is one word of the story; its height is its token ID. Repeated words land at exactly the same height, but neighbouring IDs mean nothing: *the* (6411) sits next to *that* (6409) only because they are spelled alike.

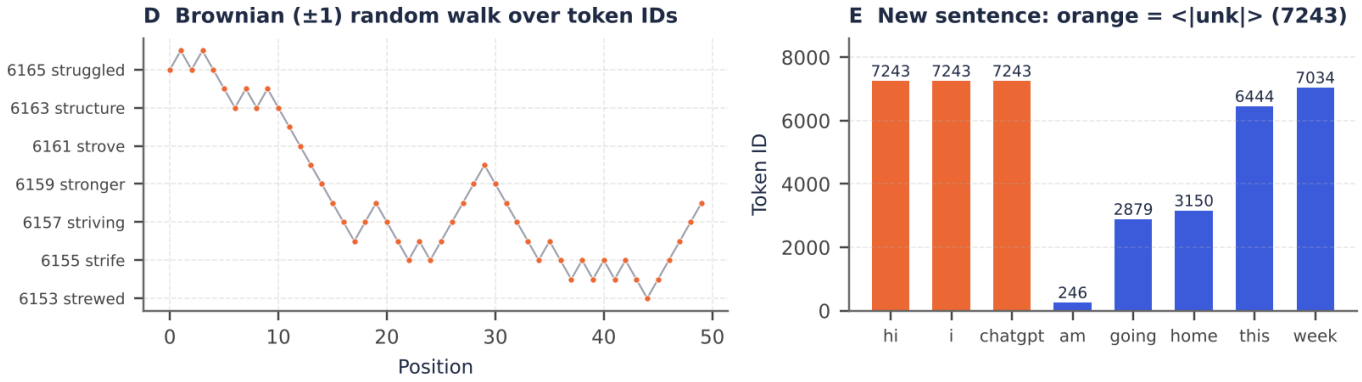
4. What the statistics show



A The running text is dominated by short function words (most common length 3), the vocabulary by longer content words (most common length 7). **B** Word counts fall roughly as 1/rank (Zipf): *the* appears 4,387 times, while 3,076 words (42% of the vocabulary) appear once. **C** The vocabulary never stops growing as more text is read (Heaps), so a word-level vocabulary has no natural size.

- **Coverage is lopsided:** the 10 most frequent words cover 26% of the text, the top 100 cover 53%, the top 1,000 cover 80%.
- **Content of the top 10** (*the, and, of, to, my, in, that, was, me, with*): function words, plus *my*/*me* from the first-person narration. Frequency alone says little about meaning.

5. Random tokens, Brownian tokens and unknown words



D A ±1 random walk over IDs decodes to *strife, strive, striving, strong, stronger ... struggle, student*: smooth in ID space because neighbours share a spelling, still meaningless. Uniform random IDs give pure word salad. Both are the zero-knowledge baseline a trained model must beat. **E** *hi* (absent from an 1818 novel), *i* (removed by the length filter) and *chatgpt* (a 2022 word) all collapse to the same `<|unk|>` ID, so the encoder survives but the round trip is lost.

6. Significance in an LLM

- **Vocabulary size sets model size.** The embedding table and the output softmax each get one row per token: 7,244 rows here, and the count would keep climbing with more books (panel C).
- **Word-level vocabularies do not scale.** Zipf means most entries are rare and poorly learned; Heaps means new text always brings new words. This is the motivation for **subword tokenizers (BPE)**, which split *chatgpt* into known pieces instead of `<|unk|>`.
- **Special tokens are part of the vocabulary.** `<|unk|>` uses the same `<|...|>` style as GPT's `<|endoftext|>`; IDs must stay contiguous because they index rows of the embedding matrix.
- **Pre-processing is part of the model.** Training and inference must clean and split identically; the `i` mismatch in step 9 is a small example of a very common bug.

7. Important things to remember (fixes made in this notebook)

Issue in the first draft	Effect	Fix
`'\d+'`, `f'[...\s]+'`	SyntaxWarning: invalid escape sequence	`r'\d+'`, `rf'[...\s]+'`
`np.zeros(int(np.max(numChars)))`	longest words (18 letters) never counted	allocate `max + 1` slots
`word2idx_new['<|unk|>'] = len(word2idx)+1`	ID 7243 skipped, hole in the ID range	`len(word2idx)` (= 7243)
`encoder(words_new, word2idx)`	`KeyError` stops "Run all"	`try/except` shows the error and continues
Brownian walk near the ends	IDs could leave `[0, nLex-1]`	`np.clip(..., 0, nLex-1)`

Takeaway: the toy tokenizer works on a real book (73,888 words, 7,243-word vocabulary, perfect round trip), but real text exposes its limits: a long tail of rare words, an ever-growing vocabulary and unseen words. Next lesson: turning these integer IDs into embedding vectors that carry meaning.